

Kafka Basics

Version 4.x

By

Anand Kulkarni

anand.kulkarni1@zensar.com

Table of Content

Module	Topic
Module 1:	Messaging Models
Module 2:	Real life scenario
Module 3:	Introduction to Apache Kafka
Module 4:	Kafka Ecosystem
Module 5:	Kafka core concepts

Importance of Data

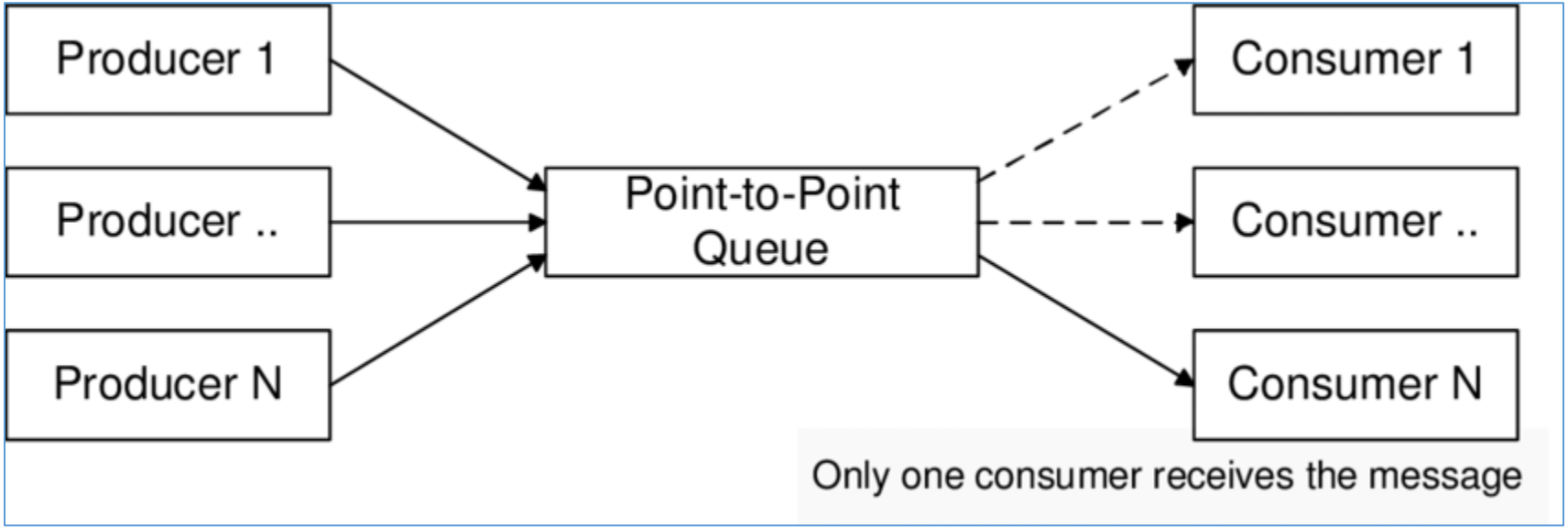
1. Every enterprise is powered by data. We take information in, analyze it, manipulate it and create more as output.
2. Every application creates data, whether it is log messages, metrics, user activity, outgoing messages, or something else.
3. The faster we can do this, the more agile and responsive our organizations can be.
4. The less effort we spend on moving data around, the more we can focus on the core business at hand. This is why the pipeline is a critical component in the data-driven enterprise.
5. How we move the data becomes nearly as important as the data itself.

Messaging Models

Messaging models are fundamental communication patterns used in distributed systems to facilitate the exchange of data and messages between various components, services, or processes. These models define how messages are sent, received, and processed within a system. There are mainly two common messaging models:

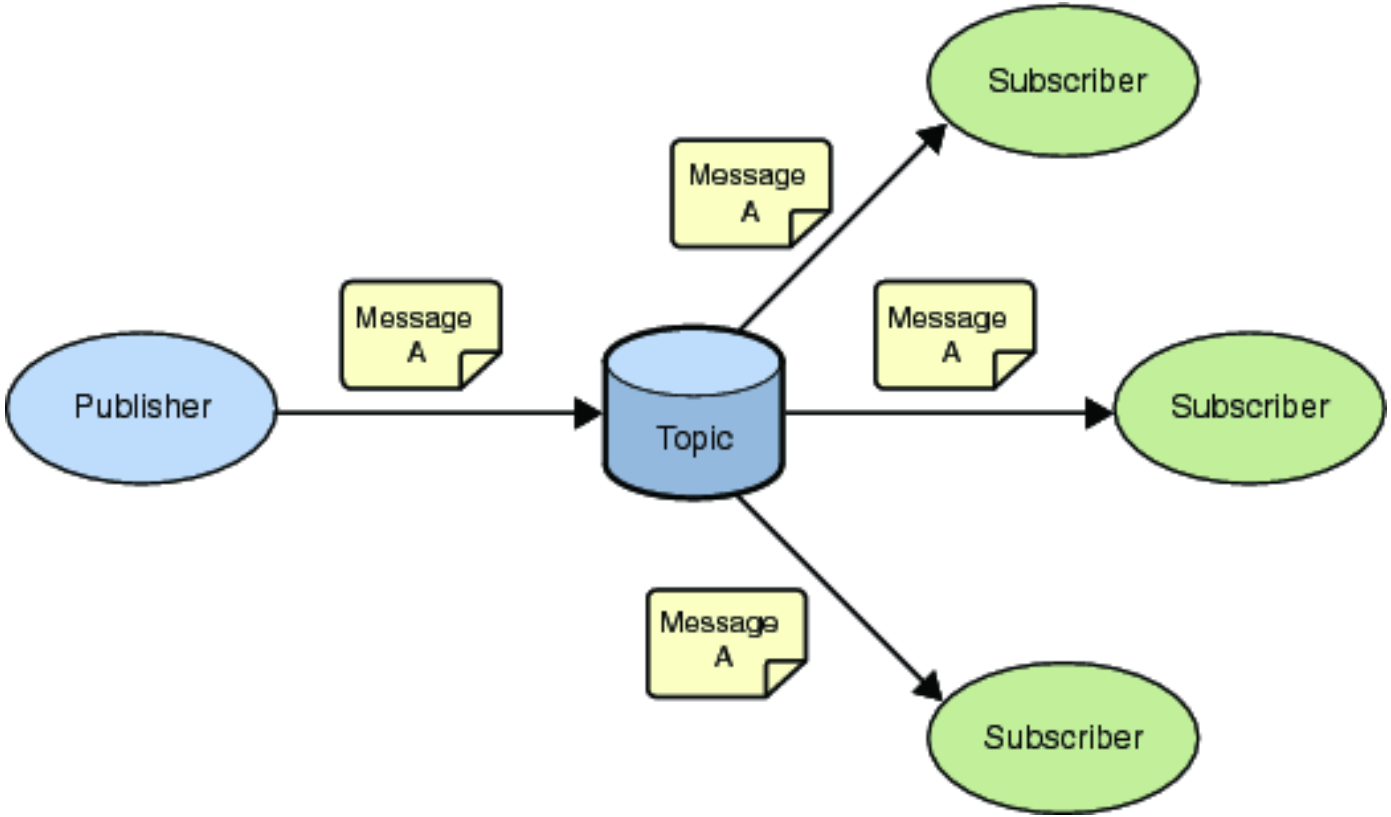
- Point-to-Point (PTP)
- Publish-Subscribe

Point-to-Point (PTP) Model



- In point to point system, a specific message can be consumed by only one consumer.
- The message will disappear from the queue once it is consumed.

Publish-Subscribe Model



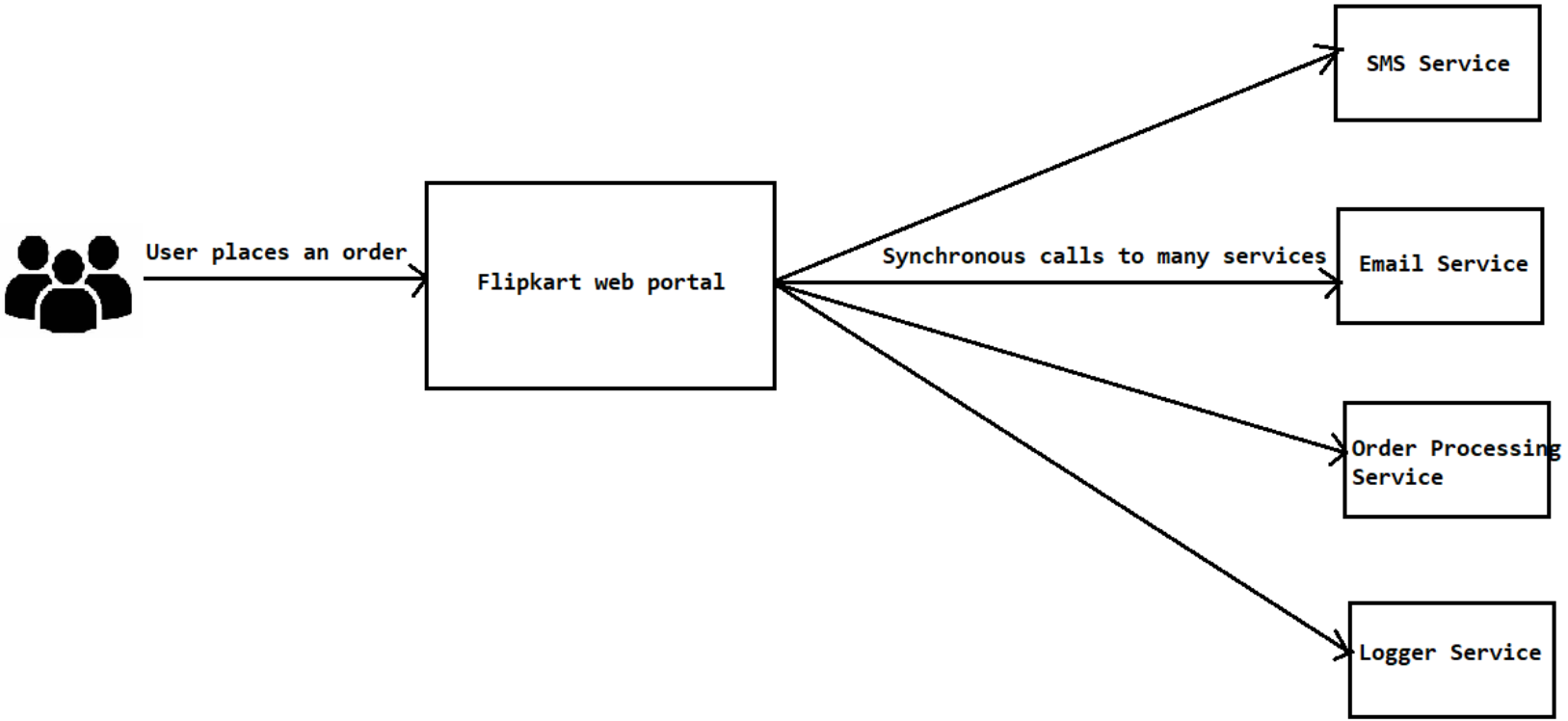
Publish-Subscribe Model

1. Publish/subscribe messaging is a pattern that is characterized by the sender (publisher) of a piece of data
2. (message) not specifically directing it to a receiver.
3. The message publisher classifies the message somehow, and that receiver (subscriber) subscribes to receive certain classes of messages.
4. Pub/sub systems often have a broker, a central point where messages are published.

Real life scenario

Flipkart, a leading e-commerce platform, offers a wide range of products and services, from electronics, furniture, books, to lifestyle, beauty, and more. A user places online purchase order. The order is accepted by Flipkart. It sends confirmation message to the user through SMS & email. Flipkart start processing the order.

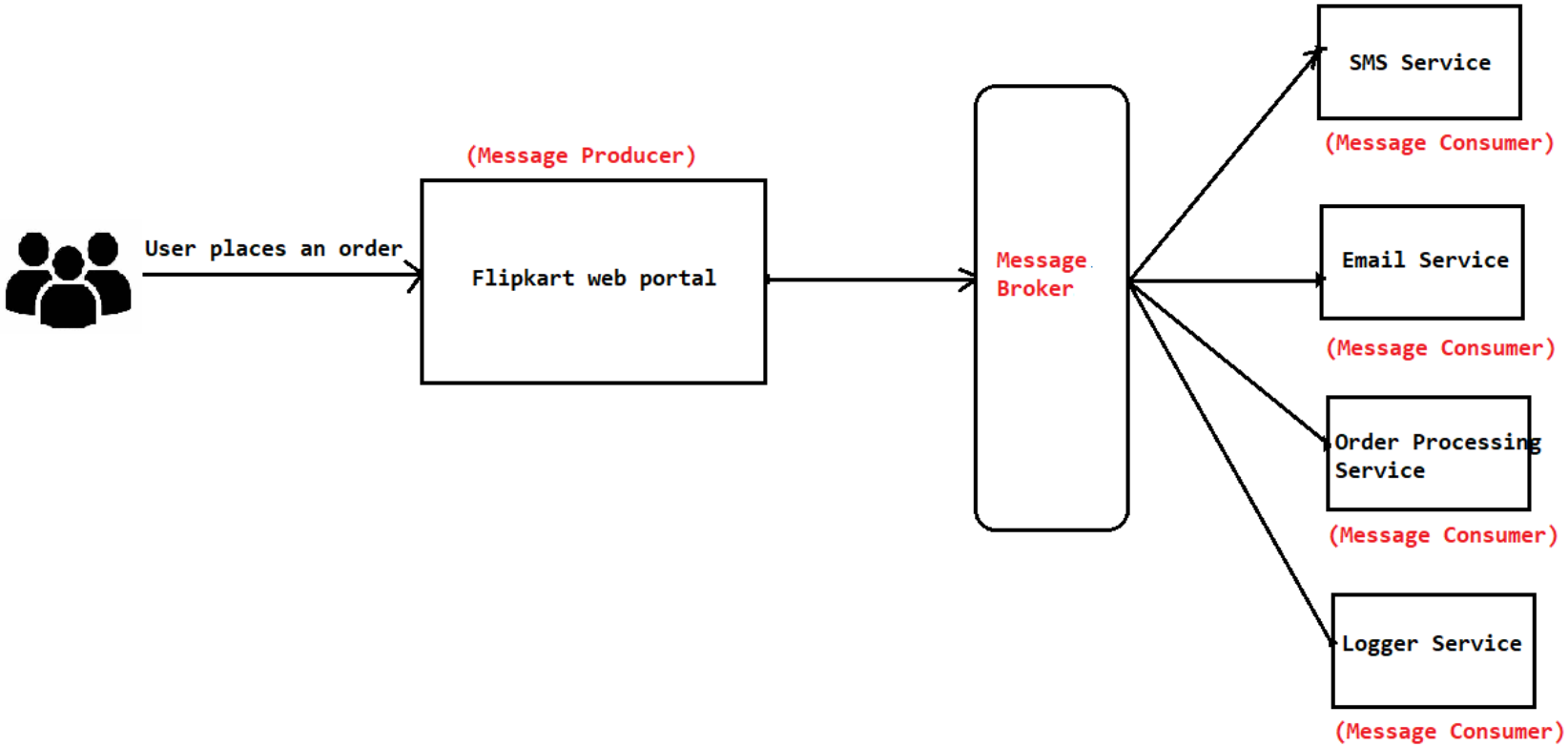
Traditional synchronous implementation



Traditional synchronous implementation

- User places an order on Flipkart. Flipkart is supposed to process the order & also need to perform other activities like logging order details, sending SMS to the customer, sending email to the customer etc.
- In traditional synchronous approach, Flipkart portal connects to several remote services like SMS, Email, Order processing, Logging etc in synchronous manner.
- The synchronous communication might lead to connectivity errors & if one of the service is down then order processing cannot be completed.

Asynchronous approach



Asynchronous approach

- In an asynchronous approach, Flipkart web portal send an 'Order' placed message to the message broker.
- Message brokers sends message acknowledgement to Flipkart portal.
- Message broker publishes 'Order' placed message to all subscribers. Thus, Flipkart portal need not concern whether message is received to subscribers or not. It is taken care by Message Broker.

Messaging System Providers

1) Java Message Service (JMS)
2) Microsoft Message Queuing (MSMQ)
3) IBM Message Queuing (IBM MQ)
4) Rabbit MQ
5) Apache Kafka

Introduction to Apache Kafka

Apache Kafka is an open-source data stream-processing software platform.

It is developed by LinkedIn. Presently it is maintained by Confluent under Apache Software Foundation.

It is a publish/subscribe messaging system & "Distributed streaming platform".

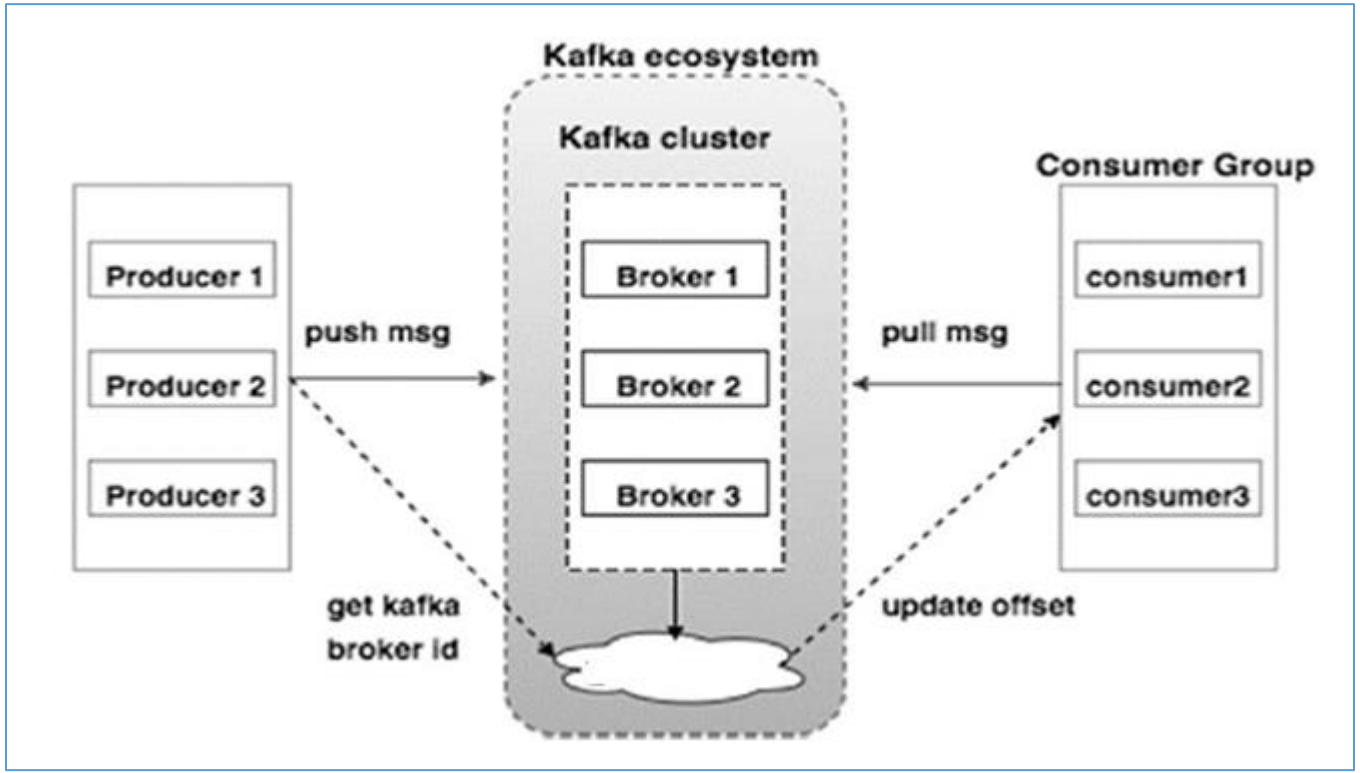
Data within Kafka is stored durably.

For better efficiency, messages are written into Kafka in batches. A batch is just a collection of messages, all of which are being produced to the same topic and partition.

Batches are also typically compressed, providing more efficient data transfer.

Kafka message is always an array of bytes.

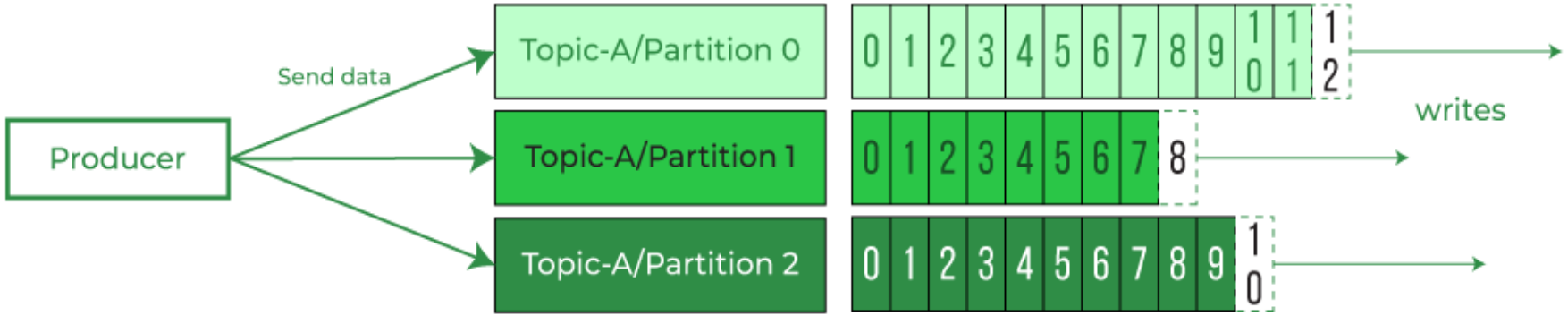
Kafka Ecosystem



Kafka core concepts

1) Topic
2) Partitions
3) Kafka Broker
4) Kafka Cluster
5) Producer
6) Consumer
7) Consumer Group
8) KRaft
9) Offset

Topic & Partitions



Topic

- A topic is a storage mechanism for a sequence of events.
- Events are immutable. Thus, we cannot change the event once it is logged into a Topic.
- Essentially, topics are durable log files that keep events in the same order as they occur in time. So, each new event is always added to the end of the log.

Partitions

- A single Kafka topic instance cannot be scalable if handles millions of messages alone. Hence, Kafka breaks a topic into nodes where each node handles subset of topic messages known as 'Partition'.
- The number of partitions is determined by us or by the cluster default configurations.
- Kafka guarantees the order of the events within the same topic partition. However, by default, it does not guarantee the order of events across all partitions.

Partitions

- To improve the ordering of events, we can set an event key to the event object. With that, events with the same key are assigned to the same partition, which is ordered. Thus, events with the same key arrive at the consumer side in the same order they were produced.

Kafka Broker

- Kafka broker allows the exchange of information between producers and consumers.
- Kafka is composed of a network of machines called brokers.
- These may not be separate physical servers but containers running on pods running on virtualized servers running on actual processors in a physical datacenter somewhere.
- Kafka broker is an Apache Kafka component that stores, manages, and distributes messages in the Kafka messaging system.
- Each broker hosts some set of partitions and handles incoming requests.
- Brokers also handle replication of partitions between each other.
- One Kafka broker can have multiple topics.

Kafka Cluster

- Kafka cluster is a set of Kafka brokers that interact with each other.
- A Kafka cluster is extremely scalable, allowing you to add or remove brokers as needed to accommodate changing data volumes or processing requirements.

Producer

- Kafka Producer is a client application that publishes (writes) events to a Kafka cluster.
- Producer sends a produce request to the leader of that partition. The partitioners shipped with Kafka guarantee that all messages with the same non-empty key will be sent to the same partition.
- Producer can also explicitly mention which partition, the message to be sent.

Consumer

- Kafka Consumer is a client application that subscribes to (reads and processes) events.
- Kafka Consumer reads messages from a Kafka Topic.
- Kafka Consumer subscribes to a specific topic & then it starts receiving the messages.
- The Kafka consumer works by issuing "fetch" requests to the brokers leading the partitions it wants to consume.

Consumer Group

- The rate of producing messages & consuming messages should match otherwise consumer will lag behind.
- In Kafka ecosystem, we can find many producers sending messages to same Kafka Topic. In such cases, we need multiple consumers to match the frequency.
- Thus, consumer group is a list of consumers subscribed to the same Topic.
- Generally, we configure number of consumers same as number of partitions of that topic, so that every consumer will consume messages from a single partition.

KRaft

- KRaft (Kafka Raft) is a consensus protocol introduced in Apache Kafka to eliminate its dependency on ZooKeeper for metadata management.
- This change simplifies Kafka's architecture by consolidating metadata handling within Kafka itself, making it more efficient and scalable.

Offset

- An offset is a unique identifier that denotes a position in a Kafka topic partition.
- When a consumer group consumes messages from a partition, it stores the offset of last message it has read from that partition, also known as consumer group's current offset.
- Imagine a Kafka topic partition as a book. The offset is like a bookmark that keeps track of the last page you have read in the book. When you resume reading the book, you can pick it up from the page marked by the bookmark. This is similar to how a consumer group uses the offset to resume consuming messages from a topic partition.
- Consumer group periodically commit its current offset to persistent storage, such as Apache Zookeeper. This ensures that the consumer group can resume from the last committed offset in case of failure.

Thank you!!